

ITA-OCR

Offline transcription of Italian handwriting

TECHNICAL REPORT

Teaching a Vision Model When to Stop: Fine-Tuning GLM-OCR for Italian Handwriting

Author: *Giorgio Ottoboni*

Model: *GLM-OCR, LoRA rank 8*

Engine: *llama.cpp, Vulkan backend*

August 2026

Contents

Abstract	3
1 Introduction	3
2 Data and split protocol	4
2.1 Composition of the corpus	4
2.2 The split contract	4
3 Training setup	6
3.1 Configuration	6
3.2 Training dynamics	6
4 The termination collapse	7
5 Isolating the cause: two stop tokens	8
6 The conditional inference cascade	9
7 Results	10
7.1 Reduction against the base model	10
7.2 The clean subset	10
7.3 The sealed benchmark	11
8 Direct OCR comparisons	12
9 Quantization	16
10 Behaviour on the processor	17
11 Reproducibility and the noise floor	18
12 Limitations and next steps	18
13 Deployment and reproduction	19
14 Conclusion	20
References	21

Abstract

ITA-OCR transcribes Italian handwritten notes entirely on the user’s machine: GLM-OCR (Z.ai, 2026) served by `llama.cpp` (Gerganov and llama.cpp contributors, 2026), with no network calls and no data leaving the device. The model was fine-tuned with low-rank adaptation (Hu et al., 2021) on 1,402 real pages written by 76 different people, and the run exposed a failure that aggregate accuracy metrics actively hide. Reading improved: character error rate fell from 21.54 to 19.84 on the cohort of pages that every model finishes. The number of pages that never emit a stop token, however, rose from 5 to 25 out of 164. The cause is that the checkpoint has *two* viable stop tokens, and the standard supervised fine-tuning template teaches the one the pretrained model never actually uses. A 2×2 experiment confirmed the mechanism; a conditional inference cascade resolved the symptom, bringing un-terminated pages down to 2 out of 164. The resulting system beats the untuned baseline on both axes, by 16.7% on character error and 16.7% on word error over the whole held-out set, and by 35.9% and 31.3% on the readable part of a sealed benchmark, at identical cost per page. It is shipped quantized to Q8_0 for both towers: 2.1 GB of video memory, 512 tokens per second of decoding, and a quality that a paired per-page comparison cannot distinguish from bf16.

1 Introduction

The target of this work is a specific and unglamorous class of document: university lecture notes, handwritten, in Italian, photographed or scanned by their authors. The pages are dense and mixed: running prose, formulas, diagrams with labels, tables sketched freehand. Every document is written by a different hand. Commercial optical character recognition handles printed Italian well and handwriting badly; general vision-language models do better, but they require sending the pages to somebody else’s servers.

These are personal documents. The corpus used here consists of notes shared by university classmates for this project, and it is not redistributable. The privacy requirement is therefore not a feature but a constraint on the architecture: everything runs on the user’s own machine. That rules out an application programming interface and forces the model to fit in consumer memory, which is what makes the sections on quantization and speed load-bearing rather than decorative.

The report is organised as follows. Section 2 describes the corpus and the protocol that governs the splits. Section 3 gives the training configuration and the loss curve. Section 4 documents the failure that fine-tuning introduced, Section 5 isolates its cause, and Section 6 describes the inference-time policy that resolves it. Section 7 reports the quality obtained against the untuned model, Section 8 shows qualitative examples from four named hands, Sections 9 and 10 the cost of running

it, Section 11 covers the reproducibility of the measurements, Section 12 what a second version should improve, Section 13 what is actually shipped, and Section 14 draws the results together.

2 Data and split protocol

2.1 Composition of the corpus

Every page is a real page of somebody’s notes, accompanied by a human transcription used as ground truth. Pages are rendered onto a fixed 960×1248 canvas at 150 DPI, which the model encodes into 1,528 image tokens. The prompt is GLM-OCR’s own **Text Recognition**; no system prompt and no custom instruction are used, so that the fine-tuned model remains directly comparable with the untuned one.

Table 1: Composition of the training and evaluation material.

Set	Pages	Writers	Gold characters
Training (after cleaning)	1,402	76	2,442,460
Development holdout	164	10	282,029
Final benchmark	103	5	sealed

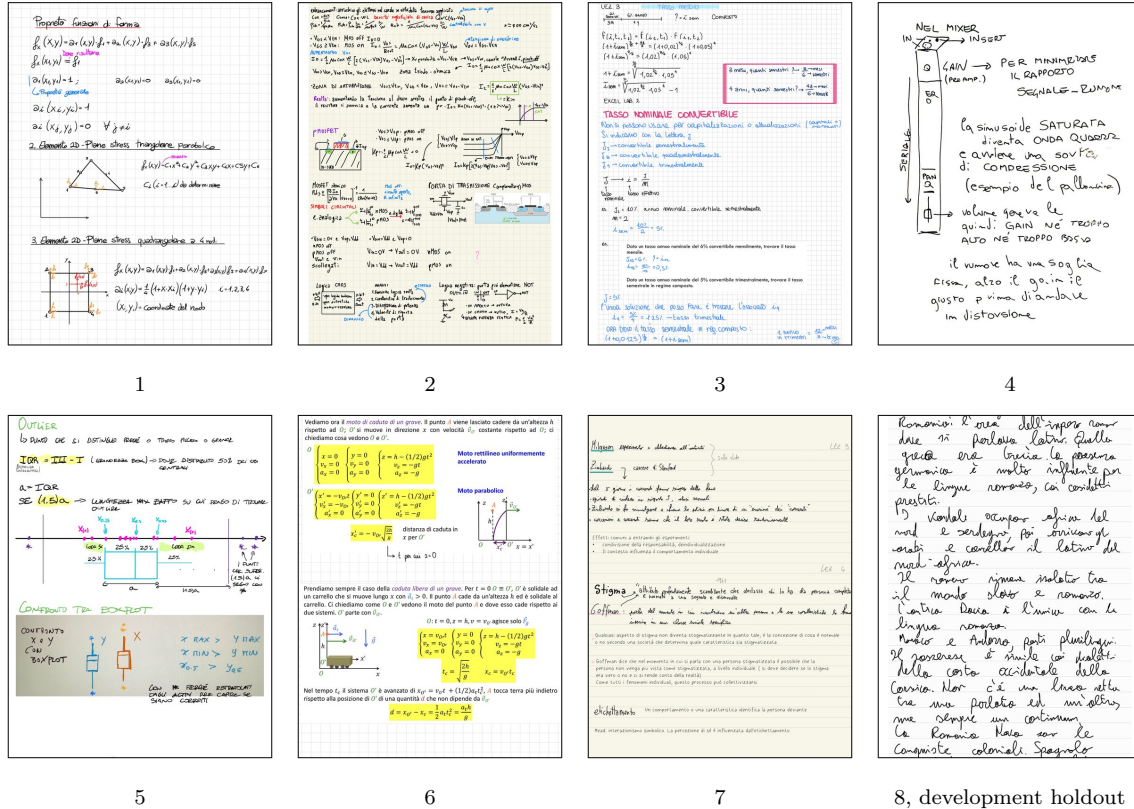
Cleaning removed four records whose transcription was verifiably partial, in each case because the printed legend of an inset figure had been omitted; eleven duplicated blank pages, since the same empty grid repeated nine times teaches the same input-to-output pair nine times; and two decorative stock covers whose ground truth was a transcription of chalkboard doodles rather than of notes.

The material is not uniform. Writers 1 to 5 and 8 wrote by hand, some on paper and some on a tablet; writers 6 and 7 contribute typed notes carrying handwritten annotation, and pages of that kind appear on the evaluation side of the split as well. A typed page teaches reading order and the notation used in the transcriptions rather than letterforms, so the count of pages overstates the amount of handwriting the model actually sees. The scarce resource here is hands, not pages: 76 of them in training, and any claim about the next user rests on how many different ways of forming a letter the model met, not on how many characters it read.

2.2 The split contract

The split is governed by a written contract rather than by a random seed, and its central rule is that **writers are disjoint across splits**. A hand that appears in training can never later become a benchmark hand, and a development-holdout writer can never be promoted into training. This matters more here than it would elsewhere, because part of what is being learned is handwriting-specific: a model evaluated on a hand it was trained on would report a number that says nothing

Figure 1: One page from each of eight writers in the corpus.



Writers 1 to 7 are in training, writer 8 in the development holdout. Pages are shown at the resolution the model receives them.

about the next user.

- **Training and validation:** 1,402 pages from 76 writers, the only material that enters the gradients.
- **Development holdout:** 164 pages from 10 unseen writers. All selection, ablation and tuning is done here.
- **Final benchmark:** 103 pages from 5 further unseen writers, sealed until the final decision. It has since been opened more than once and is therefore reported descriptively, never used as a gate.

One caveat propagates into every number in this report. The 164-page evaluation set contains 282,029 characters, but they are not evenly distributed across writers: one writer accounts for 24.4% of them and a second for 18.8%. A micro-averaged character error rate, computed as total edit distance (Levenshtein, 1966) over total characters, is therefore to the extent of 43% a measurement of two hands. Per-writer breakdowns are reported alongside the aggregates for this reason.

3 Training setup

3.1 Configuration

Fine-tuning is low-rank adaptation of the language decoder only, with the vision tower and the projector frozen, carried out with LlamaFactory (Zheng et al., 2024). The recipe stays deliberately close to the official GLM-OCR fine-tuning example; the single deviation that mattered is the subject of Section 5.

Table 2: Hyperparameters of the shipped configuration.

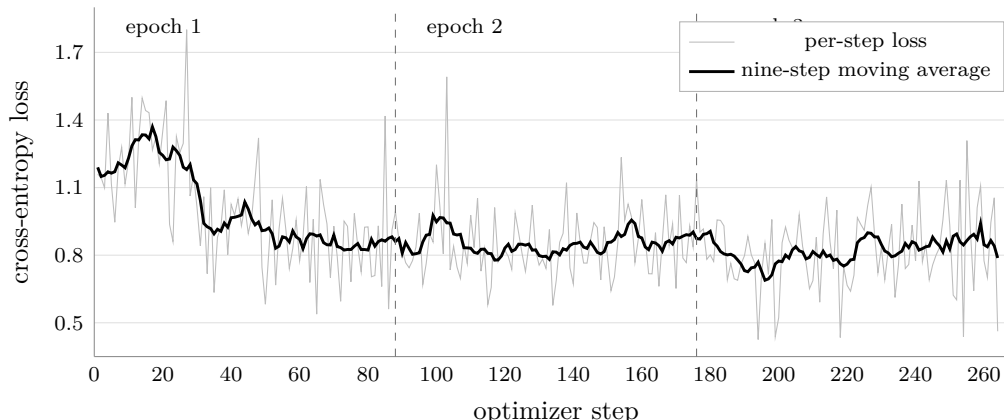
Parameter	Value
Base model	GLM-OCR
Adaptation method	LoRA, rank 8
LoRA α / dropout	16 / 0
Target modules	all, language decoder
Vision tower and projector	frozen
Precision	bf16
Sequence cutoff	8,192 tokens
Micro-batch	2
Gradient accumulation	8
Effective batch	16
Epochs	3
Optimizer steps	264
Learning rate	1.0×10^{-4}
Schedule	cosine, 10% warmup
Seed / data seed	42 / 42
Checkpoints	one per epoch

Training ran on a single Radeon RX 7900 XT under ROCm 7.2 and PyTorch 2.13. The measured profile of that card is 84.3 TFLOPS in bf16 and, on the real GLM-OCR forward and backward pass at rank 8, between 1,403 and 1,507 sequence tokens per second, reserving 8.55 GB in the worst case at roughly 270 W of median board power. A micro-batch of one already saturates the card at this rank, so larger batch shapes buy nothing. The whole run takes 58 minutes and 43 seconds for its 264 optimizer steps.

3.2 Training dynamics

The loss starts at 1.16, ends at 0.46 and averages 0.89 over the run. The endpoint is the least informative part of it. Read across the epoch boundaries instead: the smoothed curve sits at 0.86 after the first epoch and at 0.87 after the second, and only the third moves it, to 0.79. That flatness agrees with what the checkpoints score on the holdout, where the model at 88 steps reads at 17.92 character error and the one at 264 at 17.63, a difference inside the noise floor established in Section 11.

Figure 2: Training loss of the shipped run, per optimizer step, with a nine-step moving average.



Source: training log of the shipped run. Dashed lines mark the epoch boundaries.

Figure 2 is placed here, among the settings, rather than among the results, for a reason the rest of this report develops: the loss descends smoothly through the exact run in which termination collapsed. Training loss cannot see the failure this report is about.

4 The termination collapse

The first fine-tuned model read better and behaved worse. On the 164-page development holdout, the number of pages on which generation never emitted a stop token rose from 5 in the base model to 25 after fine-tuning. On such a page generation runs to the ceiling of 4,096 tokens, usually repeating itself (Holtzman et al., 2020). That is 15% of the evaluation set producing output of unbounded length.

This failure is invisible in the headline metric, and worse than invisible: it inflates it. A page that emits 4,096 tokens against a ground truth of 1,500 characters can contribute hundreds of points of character error on its own. Every table below therefore reports a *capped* character error rate next to the raw one, in which the error of each page is clipped at the length of its ground truth so that no single page can exceed 100%. Where the raw and capped figures coincide, no page blew up.

Reading quality did nevertheless improve. On the 136-page cohort that the baseline, v6 and v8 all terminate, character error fell from 21.54 to 19.84 and word error from 49.32 to 39.43. That restriction to a common cohort is the only fair comparison available, because comparing models on pages that one of them destroys measures the disaster rather than the transcription. That drop of ten points of word error is the real gain from fine-tuning, and the termination collapse was cancelling it.

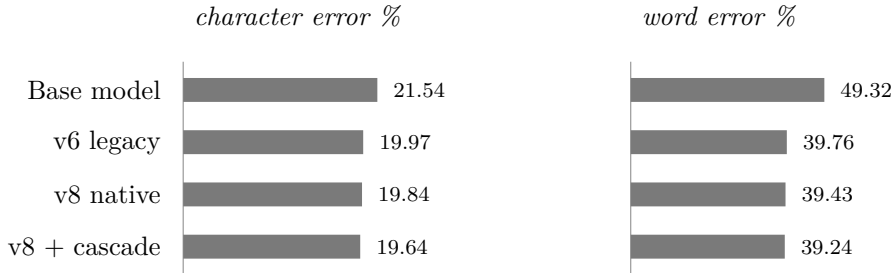
The instability is also strikingly concentrated. Of the ten held-out writers, one accounts for 17 of the 25 failures of v6 and for 16 of the 24 of v8. Whatever fine-

Figure 3: Pages that never emit a stop token, out of 164 held-out pages.



Fine-tuning multiplied the failure fivefold. Supervising the model’s native stop token, in the run labelled v8, did not repair it; the conditional cascade of Section 6, which changes no weights, brought it below the baseline.

Figure 4: Error rates on the 136 pages that every model finishes.



Lower is better. On this cohort the cascade is identical to v8 by construction: it intervenes only where greedy decoding fails, so pages that already worked come out of the same generation.

tuning damaged, it damaged for a particular kind of hand rather than uniformly.

5 Isolating the cause: two stop tokens

The generation configuration of the checkpoint accepts two end-of-generation tokens: `<|endoftext|>`, identifier 59246, which the tokenizer reports as the end-of-sequence token, and `<|user|>`, identifier 59253, the conversational turn boundary. The standard supervised fine-tuning template appends `tokenizer.eos_token_id`, that is 59246, to every gold answer. The pretrained model, left to itself, ends documents with 59253.

Fine-tuning therefore spends every example teaching the model to produce a token it had never used for this purpose, while the token it does use appears in the training data only as an unsupervised artifact. The hypothesis to be tested is

Table 3: Development holdout: 164 pages, 10 unseen writers.

Model	CER	CER cap.	WER	WER cap.	Lost
Base (no fine-tuning)	37.67	30.88	71.05	54.68	5
v6 (legacy stop, 3 epochs)	78.37	34.04	123.41	51.46	25
v8 (native stop, 3 epochs)	76.26	34.81	118.25	51.79	24
v8 with cascade	28.49	25.71	49.67	45.54	2

that this is not catastrophic forgetting in general, but the destruction of one specific behaviour: knowing that the page is over.

The test was a 2×2 design of one epoch per cell, holding everything else identical: the supervised terminator, legacy 59246 against native 59253, crossed with the learning rate, 1×10^{-4} against 3×10^{-5} .

Table 4: Supervised terminator crossed with learning rate, 88 steps per cell, development holdout.

Configuration	CER capped	WER capped	Pages lost
Base	30.88	54.68	5
Native stop, lr 3×10^{-5}	35.17	56.72	18
Native stop, lr 1×10^{-4}	35.03	54.14	21
Legacy stop, lr 3×10^{-5}	38.08	57.90	23
Legacy stop, lr 1×10^{-4}	38.53	56.86	34

The ordering is monotone on both factors: the native terminator beats the legacy one at equal learning rate, and the lower learning rate helps within each terminator. The mechanism then shows itself directly in the generations. Across all four cells, free generation stops only on `<|user|>`, and token 59246 never appears in any stream, not even in the arms trained specifically to emit it. The model can be taught to raise the probability of that token and still never sample it.

This result decided the architecture. Supervising the correct stop token improves the ordering but does not solve the problem: v8, trained on the native terminator for three epochs, still loses 24 pages out of 164 against the 5 of the baseline. Fine-tuning made this model a better reader and a worse finisher, and no choice of learning rate or terminator that was tested reverses that. The repair had to move to inference.

6 The conditional inference cascade

Rather than fight the training dynamics, the deployed system detects the failure and retries. A panel of 48 pages, the 24 that the fine-tuned model loses together with 24 controls that terminate normally, was used to choose the retry parameters, measuring not only how many failures each setting recovers but how many working pages it breaks.

The frequency penalty is the component that works; the presence penalty alone and DRY sampling do not. The final policy has three stages. Greedy decoding comes first; if the page does not terminate or enters a repetition loop, it is retried with a frequency penalty of 0.35 and a presence penalty of 0.20; if it still loops, it is retried with the frequency penalty alone. If that also fails, the prefix preceding the loop is returned, explicitly marked incomplete rather than silently truncated.

The last column of Table 5 is load-bearing. On penalized generations there are pages that loop for a while and then stop correctly, so the loop detector must

Table 5: Inference-time mitigations measured on the 48-page panel.

Configuration	Pages lost	Recovers	Breaks	Loops
Greedy (no mitigation)	24	n/a	n/a	0
Frequency 0.35 + presence 0.20	5	20 / 24	1	1
Frequency 0.35	5	21 / 24	2	5
Frequency 0.20 + presence 0.10	8	19 / 24	3	8
Presence 0.20 alone	22	4 / 24	2	0
DRY sampling	22	2 / 24	0	0

interrupt only where a further stage remains to be run. Aborting the final stage converts recoverable pages into lost ones, which a control run confirmed: it lost 4 pages instead of 2.

Table 6: Behaviour of the cascade on the two evaluation sets.

Set	Pages	Greedy	Retry 1	Retry 2	Incomplete
Development holdout	164	140	19	3	2
Final benchmark	103	71	21	3	8

The cascade is cheaper than not having it. On the holdout it takes 512 seconds of wall clock against the 625 seconds of a plain greedy pass over the same pages. Pages in runaway used to be carried all the way to the ceiling of 4,096 tokens; the detector now cuts them at around 550 tokens, and the retry that follows is short. Three passes over the worst 15% of the pages cost less than one pass that lets them run.

7 Results

7.1 Reduction against the base model

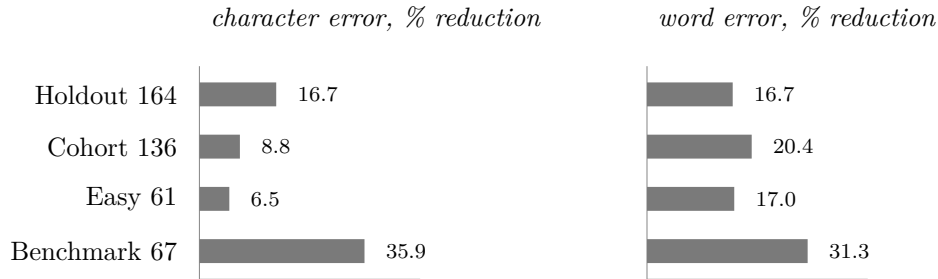
Against the untuned model the shipped system is better on every evaluation set available, on both axes, and by margins far larger than the noise floor of 0.3 points established in Section 11.

In absolute terms the holdout moves from 30.88 to 25.71 capped character error and from 54.68 to 45.54 capped word error, and the readable part of the benchmark from 29.28 to 18.78 and from 56.74 to 38.99.

7.2 The clean subset

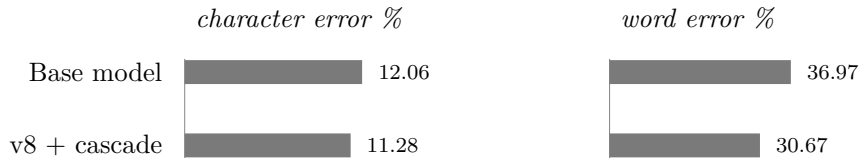
The fairest single read on the gain from fine-tuning is the 61-page subset whose difficulty is declared *easy* in the manifest, an a priori label attached to the document rather than a selection made on the outcome. Neither model loses a page here, raw and capped error coincide, and the cascade never fires, so this compares transcription against transcription with nothing else mixed in.

Figure 5: Relative reduction of error against the base model, by evaluation set.



Higher is better, and every bar is a win. The holdout row uses capped character error, the only fair statistic where pages are lost; on the other three sets neither model loses a page, so raw and capped coincide. The benchmark row excludes the single writer discussed in Section 7.3.

Figure 6: The 61 pages of declared easy difficulty: base model against the shipped system.



52 pages come from the development holdout and 9 from the sealed benchmark. Word error falls by 17.0% in relative terms and character error by 6.5%.

Per page, the fine-tuned system wins on 43, loses on 17 and ties on 1. The aggregate advantage is real, and it is not uniform.

Inference cost is unchanged by fine-tuning: 1.61 seconds per page for both the base and the fine-tuned model on this subset, 383 against 382 tokens per second of decoding, and about 297 generated tokens per page. The prompt is 1,542 tokens, almost all of it the image, so end-to-end throughput is roughly 185 tokens per second against the 383 of pure generation.

7.3 The sealed benchmark

On the sealed 103-page benchmark the picture is the same once one hand is separated out. A third of that set is a single writer at the edge of legibility, whom the base model already reads at 52% capped character error; the aggregate over all 103 pages says more about that hand than about any model. On the remaining 67 pages the shipped system loses no page at all and beats the base model on both axes. The unmitigated fine-tuned model, on those same 67 pages, was worse than the baseline and lost 4.

8 Direct OCR comparisons

Aggregate rates do not show what a user actually receives. Each comparison therefore pairs an enlarged input region with verbatim excerpts from the base OCR, raw v8 and the deployed v8 cascade. The pages come from single-writer folders outside the mixed-writer directory and are labelled only Examples 1–4. Examples 1, 3 and 4 show training fit; Example 2 is from an unseen development-holdout writer. Writer-disjoint evidence remains in Section 7.

Example 1 — equations and eigenvectors (training)

$\hookrightarrow \lambda_2 = 1359 \cdot 10^6 \left(\frac{\text{rad}}{\text{s}}\right)^2 \rightarrow \omega_2 = \sqrt{\lambda_2} = 1166 \text{ rad/s}$
 $f_2 = \frac{\omega_2}{2\pi} = 185 \text{ Hz}$
 $\hookrightarrow \lambda_3 = 2241 \cdot 10^5 \left(\frac{\text{rad}}{\text{s}}\right)^2 \rightarrow \omega_3 = \sqrt{\lambda_3} = 473,44 \text{ rad/s}$
 $f_3 = \frac{\omega_3}{2\pi} = 75 \text{ Hz}$

$\{[K] - \lambda [I]\} \cdot \{f\} = 0$
 Considero $\lambda_1 = 0$

$\begin{bmatrix} K & -K & 0 \\ -K & 2K & -K \\ 0 & -K & K \end{bmatrix} \begin{Bmatrix} f_1' \\ f_2' \\ f_3' \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \\ 0 \end{Bmatrix}$

$Kf_1' - Kf_2' = 0 \Rightarrow f_1' = f_2'$
 $-Kf_2' + Kf_3' = 0 \Rightarrow f_2' = f_3'$

$\{f'\} = \begin{Bmatrix} 1 \\ 1 \\ 1 \end{Bmatrix} \quad f_2' = 1$

Ortogonalità degli autovettori
 - prodotto scalare $\{f'\}^t \cdot \{f''\} = 0$
 - condizione generale $\{f'\}^t [M] \{f''\} = 0$

Figure 7: Example 1, enlarged around the eigenvector calculation discussed below.

Table 7: Example 1: direct OCR comparison. Excerpts are verbatim.

System	Output excerpt or cascade action	CER	EOS
Ground truth	$\{f'\} = [1, 1, 1]^t$. <i>Ortogonalità degli autovettori.</i>	n/a	n/a
Base OCR	$\{f'\} = [1, 1]^t$. <i>Ortogonalità degli autovettori.</i>	55.93%	yes
v8, greedy	$\{f'\} = [1, 1, 1]^t$. <i>Ortogonalità degli autovettori.</i>	26.99%	yes
v8 + cascade	Identical to v8 greedy: the first attempt emitted EOS, so no retry was run.	26.99%	yes

The base loses an eigenvector component and corrupts the technical noun; v8 restores both. Since v8 emits EOS normally, the cascade returns the same output.

Example 2 — continuous cursive (development holdout)

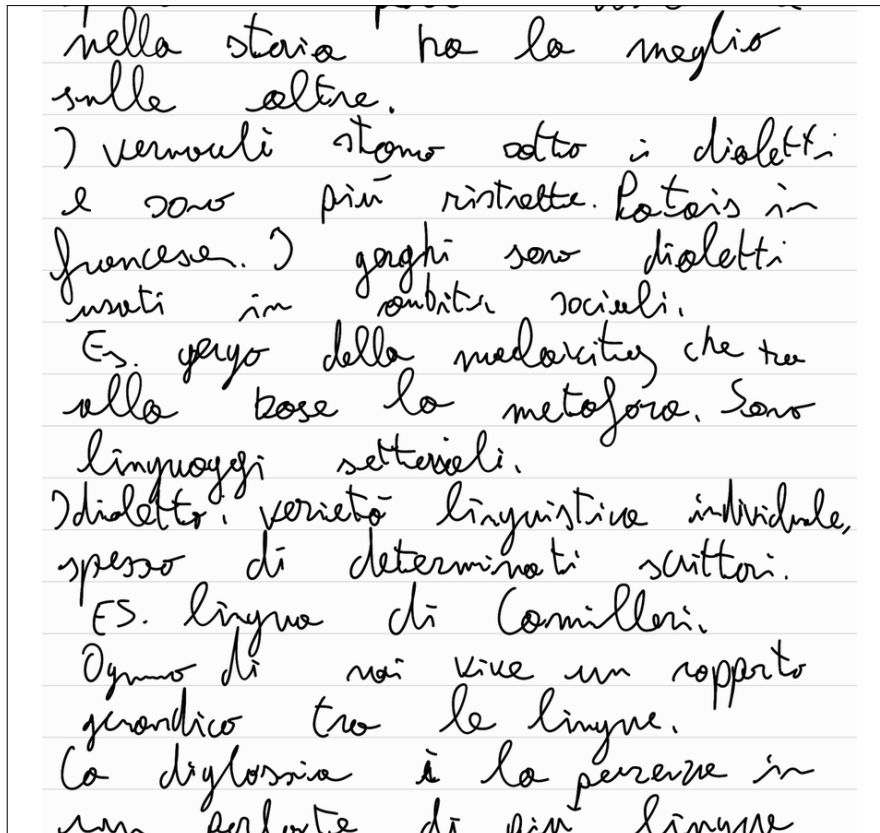


Figure 8: Example 2, enlarged around the continuous-cursive passage compared below.

Table 8: Example 2: direct OCR comparison. Excerpts are verbatim.

System	Output excerpt or cascade action	CER	EOS
Ground truth	<i>Sono linguaggi settoriali. Idioletto: varietà linguistica individuale.</i>	n/a	n/a
Base OCR	<i>Sono linguaggi settvereli.) dialetto, verieto linyustive individuale.</i>	17.33%	yes
v8, greedy	<i>Sono linguaggi settentorii.) dialetto, varietà linguistica individuale.</i>	14.11%	yes
v8 + cascade	Identical to v8 greedy: the first attempt emitted EOS, so no retry was run.	14.11%	yes

The fine-tuned model does not make the page error-free, but it recovers *linguaggi* and the complete phrase *varietà linguistica individuale*. This is also the only example here written by a person absent from training.

Example 3 — vector notation and a diagram (training)

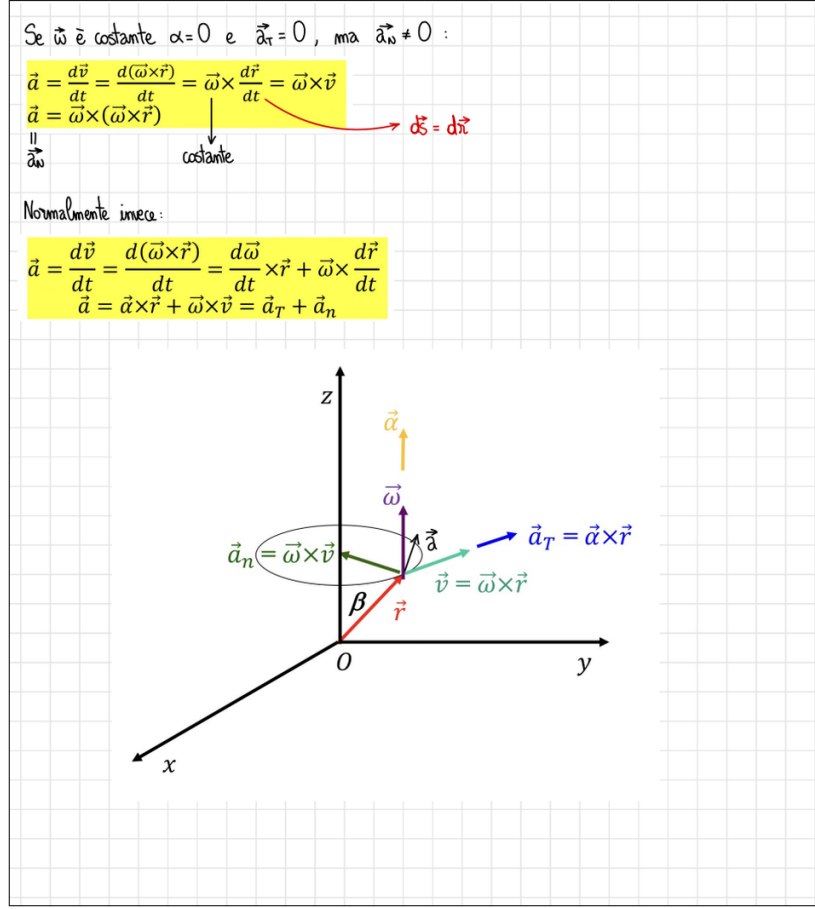


Figure 9: Example 3, enlarged around the equations and labelled diagram.

Table 9: Example 3: direct OCR comparison. Excerpts are verbatim.

System	Output excerpt or cascade action	CER	EOS
Ground truth	<i>Se $\vec{\omega}$ è costante $\alpha=0$ e $\vec{a}_T=0$, ma $\vec{a}_N \neq 0$.</i> Labels: z , $\vec{\alpha}$, $\vec{\omega}$, β , $\vec{r}$.	n/a	n/a
Base OCR	<i>Se $\vec{\omega}$ è costante $\alpha=0$ e $\vec{a}_T=0$, ma $\vec{a}_N \neq 0$.</i> Several diagram labels are absent.	35.89%	yes
v8, greedy	<i>Se $\vec{\omega}$ è costante $\alpha=0$ e $\vec{a}_T=0$, ma $\vec{a}_N \neq 0$.</i> The labels z , $\vec{\omega}$, β and $\vec{r}$ are retained.	13.89%	yes
v8 + cascade	Identical to v8 greedy: the first attempt emitted EOS, so no retry was run.	13.89%	yes

Here the largest difference is structural. The base OCR changes the angular velocity symbol and drops labels around the diagram; v8 preserves the notation and more of the reading order.

Example 4 — missing-EOS hallucination

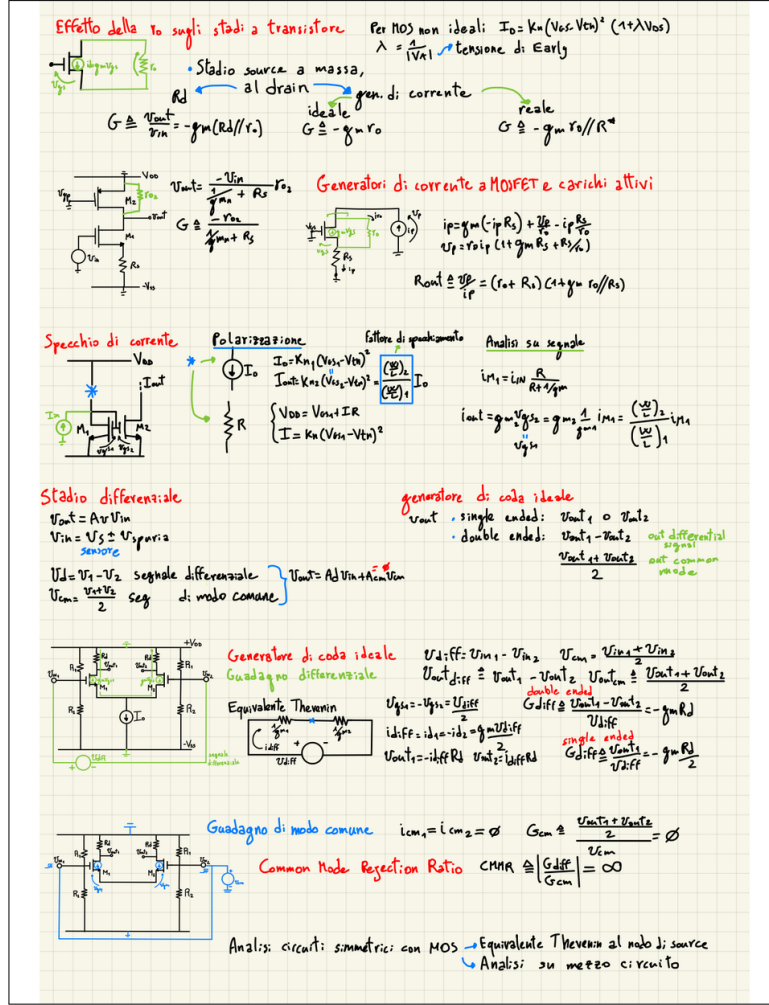


Figure 10: Example 4 as supplied to the model. The page contains several circuit diagrams but no repeated block of text.

Table 10: Example 4: base OCR, raw v8 and v8 with the deployed cascade.

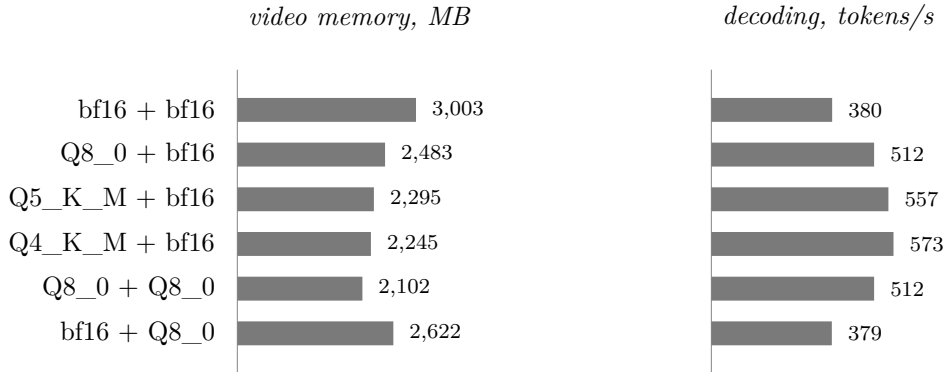
System	Output excerpt or cascade action	CER	EOS
Ground truth	Generatori di corrente a MOSFET e carichi attivi ... Specchio di corrente ... Stadio differenziale.	n/a	n/a
Base OCR	Generatori di corrente a MOIFET e carichi attivi ... Fattore di spacciamente ... mezzo circuito.	45.64%	yes
v8, greedy	After the heading, repeats $G \triangleq V_{out} = -V_{in} / (1/g_m + R_s) V_{in}$ until the 4,096-token limit.	291.15%	no
v8 + cascade	Generatori di corrente a MOFET e carichi attivi ... Specchio di corrente ... Stadio differenziale.	40.71%	yes

This final comparison is the reason to distinguish raw v8 from the shipped system. Raw v8 produces 7,659 characters and reaches 4,096 tokens without an EOS. The cascade recognises a 29-token span repeated eight times, interrupts the first attempt at token 340, and retries with frequency penalty 0.35 and presence penalty 0.20. The retry emits the native stop token 59253. The cascade spends 1,408 tokens across both attempts and returns 2,183 characters, close to the 2,229-character reference. It therefore converts the hallucinated 291.15% CER into 40.71%, slightly better than the base OCR, without changing the fine-tuned weights.

9 Quantization

The language model was quantized from bf16 with `llama-quantize`, and the vision tower re-converted from the original checkpoint at `q8_0`. All variants were evaluated on the same 164 pages under the same cascade policy, and compared on the clean cohort of 126 pages that every configuration closes on the greedy pass, which excludes the retry path and with it the high-variance part of the measurement.

Figure 11: Video memory and decoding speed across quantizations.



Quantizing the language model buys most of the speed; quantizing the vision tower buys memory and no speed, because the image encoder is not what the decoding loop waits on.

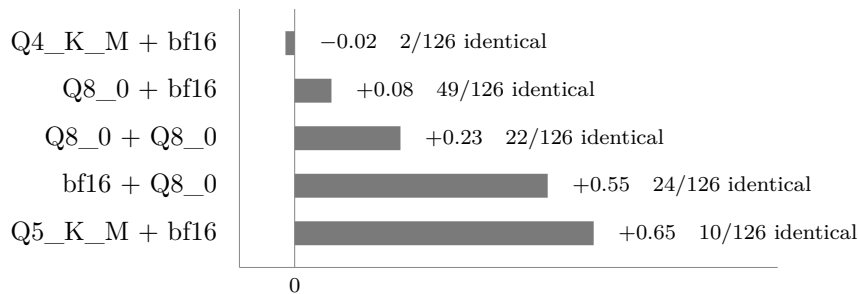
Table 11: Quantization on the graphics processor, 126-page clean cohort.

Language model + tower	CER	WER	s/page	Decoding	Memory
bf16 + bf16	17.94	37.98	3.00	380 tok/s	3,003 MB
Q8_0 + bf16	18.30	38.44	2.44	512 tok/s	2,483 MB
Q5_K_M + bf16	18.73	38.72	2.48	557 tok/s	2,295 MB
Q4_K_M + bf16	17.71	37.85	2.36	573 tok/s	2,245 MB
Q8_0 + Q8_0	17.99	38.11	2.47	512 tok/s	2,102 MB
bf16 + Q8_0	18.80	38.79	2.91	379 tok/s	2,622 MB

Read on micro-averaged character error alone, Table 11 appears to say that Q4_K_M is the best configuration, better than bf16 itself. That is an artifact of

the averaging: the micro average weights each page by its length, and two long pages finishing well move the aggregate by half a point. The honest comparison is paired, page by page, against the bf16 output.

Figure 12: Mean per-page difference in character error against the bf16 reference, 126-page cohort.



Zero is bf16 itself; positive is worse. Beside each bar is the number of pages that came out character-identical to bf16.

Q8_0 is the only variant that is genuinely indistinguishable: a mean difference of +0.08, a median of zero, and a count of better and worse pages close to a coin toss. Q5_K_M and Q4_K_M show a small but directional degradation, with roughly 60% of pages worse and almost no page left untouched: 49 pages out of 126 are identical to bf16 under Q8_0, against 2 out of 126 under Q4_K_M.

What ships is therefore Q8_0 for both towers. Against bf16 it uses 30% less video memory, decodes 35% faster, and is paired-identical in quality. Q4_K_M would add 27 tokens per second and save 143MB in exchange for a degradation that is small but consistently in the wrong direction and, as the next section shows, it does not help where the time actually goes.

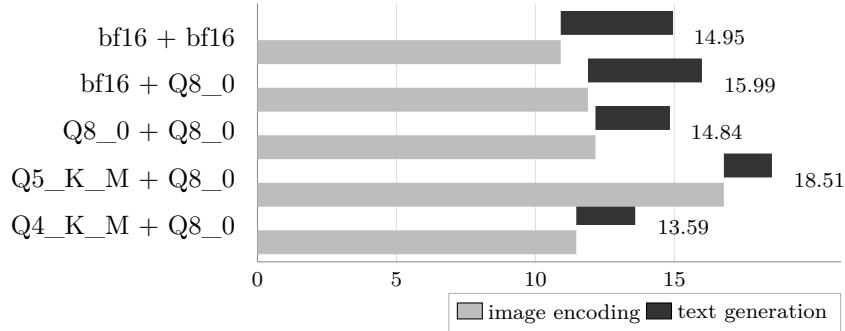
10 Behaviour on the processor

The same model on the processor alone, with the vision tower kept off the graphics card, tells a different story. Decoding nearly doubles from bf16 to Q4_K_M, from 63.8 to 124.2 tokens per second, and generation time drops from 4.0 to 2.1 seconds. Yet the page still costs about fourteen seconds, because encoding the image takes eleven to twelve of them.

On the processor, then, quantization buys memory rather than latency: 3,410 MB resident at bf16 against 2,402 MB at Q8_0. The lever for latency would be reducing the number of tokens per image, that is the resolution of the canvas, not the precision of the weights.

One deployment detail deserves to be stated plainly, because it dwarfs every saving discussed above. The inference server keeps a prompt cache in host memory with a default ceiling of 8,192 MiB, and fills it even when every request disables prompt caching. In this workload every page is a different image, so the cache never

Figure 13: Seconds per page on the processor, split between encoding the image and generating the text.



The encoder has 1,528 image tokens to grind through and is compute-bound, indifferent to how many bits the weights carry. Three pages per configuration: the trustworthy columns are the decoding rate and the resident memory, which depend on the weights rather than on the scheduler.

hits and the memory is simply wasted. The evidence that this has nothing to do with the weights is that resident memory stayed between 10.3 and 10.5 GB across all six configurations of Table 11, identical from bf16 to Q8_0 with a quantized tower; with the cache disabled it falls to between 2.3 and 3.4 GB. Anyone packaging this model must pass that flag, or ask its users for 10 GB of memory to hold a cache that will never be read.

11 Reproducibility and the noise floor

Re-running the same greedy generation, with the same weights, temperature zero, seed zero and prompt caching disabled, does not always produce the same text on the Vulkan backend. It is not an artifact of streaming; the non-streaming path behaves identically.

The instability is bounded and, more importantly, it does not change outcomes. Across two runs a day apart, the set of unterminated pages was exactly the same 24 of 164, and 136 of the 140 completed texts, or 97.1%, were character-identical. The effect on the aggregates is about 0.2 points of character error. Two operational rules follow, and both are applied throughout this report: verification by identity of the text is not a usable criterion, and aggregate differences below roughly 0.3 points of character error are not interpreted.

12 Limitations and next steps

What is described here is a first version, and it already does the job it was built for: it reads unseen Italian handwriting better than the untuned model on every set measured, it finishes more pages than the untuned model, and it runs offline on ordinary hardware. The points below are the ones a second version should pick up. None of them is a dead end, and for each there is a concrete route forward.

- **Termination is repaired at inference rather than in the weights.** The cascade brings unterminated pages from 24 to 2 out of 164, which is fewer than the untuned baseline loses, and it costs less than a plain greedy pass. Repairing the behaviour during training as well would remove the retry path altogether. The experiment of Section 5 already indicates the direction: the native terminator at the lower learning rate is the best cell measured, and it was tested for a single epoch only.
- **The advantage is real in aggregate and uneven page by page.** On the clean subset the system wins on 43 pages, ties on 1 and loses on 17. The aggregate is better on every set measured; narrowing the per-page spread is a question of corpus breadth rather than of method.
- **Scale of the corpus.** 76 writers proved enough to train a model that generalises to hands it has never seen. More writers, and more pages for each writer, is the cheapest improvement available and the one most likely to move the per-page spread.
- **Latency on the processor is bounded by the image encoder.** Eleven to twelve seconds of a fourteen-second page are spent encoding the image, and no choice of weight precision touches that. Reducing the resolution of the canvas, and with it the number of image tokens, is an untested lever that leaves the language model untouched.
- **Timings on the processor rest on three pages per configuration.** The decoding rate and the resident memory are solid; the per-page latency deserves a longer run before it is quoted as a specification.

Every one of these is addressable without changing the architecture, which is the useful thing to be able to say at the end of a first version.

13 Deployment and reproduction

The system ships as a desktop application with a Rust core, driving the inference server as a local subprocess and selecting CUDA, Vulkan or the processor according to the hardware, in that order. Two model files totalling 1.15 GB, the Q8_0 language model at 686 MB and the Q8_0 vision tower at 462 MB, are distributed alongside it, together with lexical post-correction against Italian and English dictionaries. No network call is made at any point.

Every number in this report is regenerated from the evaluation artifacts rather than transcribed by hand, and each artifact is recorded with a truncated SHA-256 digest in the repository. The training recipe consists of the configuration of Table 2 together with a patch that registers the native-stop template, and the pinned

upstream commits of the inference engine are carried as submodules, so that a fresh clone reproduces the build and, given the data, the run itself.

14 Conclusion

A vision-language model of general purpose was adapted, on a single consumer graphics card and in under an hour of training, into a specialised reader of Italian handwriting that is better than its starting point on every measurement taken.

Table 12: Summary of the shipped system against the untuned base model.

Measure	Result
Character error, readable benchmark	−35.9%
Word error, readable benchmark	−31.3%
Character error, full holdout	−16.7%
Word error, full holdout	−16.7%
Word error, common cohort	−20.4%
Word error, clean subset	−17.0%
Pages left unterminated, of 164	2, against 5 for the base model
Time per page, graphics card	1.61 s, unchanged from the base model
Decoding rate	512 tokens/s
Video memory	2,102 MB
System memory, processor only	2,402 MB
Model files shipped	1.15 GB
Network calls	none

Three results are worth restating. The first is that the largest improvement appears exactly where a reader would want it, on the readable part of a sealed benchmark written by people the model had never seen, where character error falls by 35.9% and word error by 31.3%. The second is that the improvement is free at runtime: a page costs 1.61 seconds with the fine-tuned model and 1.61 seconds without it, so the quality is obtained without asking the user to wait longer. The third is that the system finishes more pages than the model it started from, 2 unterminated out of 164 against 5, which means the failure documented at length in this report ends up on the right side of the ledger.

The cost of running it is modest by design. On a graphics card the shipped quantization occupies 2,102 MB of video memory, 30% less than the unquantized weights, while decoding 35% faster and remaining indistinguishable from them in a paired per-page comparison. Without a graphics card at all, the same models run in about 2.4 GB of system memory, which places the system within reach of an ordinary laptop rather than of a workstation. The two model files together weigh 1.15 GB and are distributed with the application, and no part of the pipeline opens a network connection: the pages, which are personal documents, never leave the

machine they were scanned on.

Finally, the measurements themselves are meant to survive scrutiny. Every figure in this report is regenerated from stored evaluation artifacts rather than transcribed by hand, each artifact is recorded with a digest, the splits are disjoint by writer under a written contract, and a measured noise floor tells the reader which differences are worth interpreting and which are not. That discipline is what makes the failure described in Section 4 visible at all, and it is the part of this work that transfers to the next version most directly.

References

- Gerganov, G. and llama.cpp contributors (2026). llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>. Software, pinned at commit 4df29be.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10(8):707–710.
- Z.ai (2026). GLM-OCR. <https://huggingface.co/zai-org/GLM-OCR>. Vision-language model checkpoint.
- Zheng, Y., Zhang, R., Zhang, J., Ye, Y., Luo, Z., and Ma, Y. (2024). LlamaFactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*.